

Курс «Введение в Linux»

Раздел 3 — vim, bash-скрипты, sed, gnuplot

Иванова Анастасия Сергеевна

2026-05-16

1. Докладчик

2. Выводы

Раздел 1

1. Докладчик

1. Докладчик

- Иванова Анастасия Сергеевна



1. Докладчик

- Иванова Анастасия Сергеевна
- 1 курс группа НКАбд-07-25



1. Докладчик

- Иванова Анастасия Сергеевна
- 1 курс группа НКАбд-07-25
- Российский университет дружбы народов



1. Докладчик

- Иванова Анастасия Сергеевна
- 1 курс группа НКАбд-07-25
- Российский университет дружбы народов
- 1132250427@rudn.ru



Вопрос 1. Как выйти из vim

Ответ: : затем q затем Enter

Почему так: В vim выход через командный режим: Esc, потом :, потом q, потом Enter.

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

✓ Правильно, молодец!

- ☐ "q"
- ☐ ":", затем "q"
- ☐ "Q"
- ☐ "Ctrl", затем "x"
- ☒ ":", затем "q", затем "Enter"

Следующий шаг

Решить снова

Рисунок 1: Вопрос 1

Вопрос 2. Перемещение по словам в vim

Ответ: - В этой строке 5 «больших слов» (WORD) - В этой строке 9 «слов» (word) - Нажимая только на w, нельзя переместить курсор на «.» - Чтобы попасть в конец строки, нужно меньше нажатий на W, чем на w - Нажимая только на W, нельзя переместить курсор на «.»

Почему так: w — по словам (буквы, цифры, _). W — по большим словам (разделитель — только пробел).

При перемещении в vim "по словам" есть небольшая разница в том, используем мы маленькую (w, e, b) или большую (W, E, B) букву. Первые перемещают нас по "словам" (word), а вторые по "большим словам" (WORD). Посмотрите справку по этим перемещениям и разберитесь в чем заключается разница между word и WORD.

А для того, чтобы убедиться, что вы разобрались, отметьте ниже **все верные** утверждения про следующую строку:

```
strange_ TEXT is_here. 2+2 YES!
```

Примечание: во всех утверждениях имеется ввиду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Подсказка: чтобы вызвать **vim-справку** по, например, перемещению 'w', нужно открыть vim и ввести команду `:help w`. Вы попадете в место справки, где описано это перемещение, а так как все перемещения описаны рядом, то двигаясь по тексту вверх и вниз можно прочитать и про 'e' и про 'b' и, самое главное, про word и WORD. Кроме того, можно вызвать сразу справку по термину word при помощи `:help word`. Чтобы закрыть справку, нужно ввести команду `:q`.

Выберите все подходящие ответы из списка

☒ Все правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментарии](#), отвечая на их вопросы, или сравнить своё решение с [другими](#) на [форуме решений](#).

- ☒ В этой строке 5 "больших слов" (WORD)
- ☒ В этой строке 9 "слов" (word)
- ☐ Нажимая только на w, нельзя переместить курсор на "."
- ☒ Чтобы попасть в конец строки, нужно совершить меньше нажатий на W, чем на w
- ☒ Нажимая только на W, нельзя переместить курсор на "."
- ☒ После 10 нажатий на W курсор окажется там же, где бы он был после 10 нажатий на w

Вопрос 3. Редактирование строки в vim

Вопрос: Как превратить «one two three four five» в «three four four four five»?

Ответ: d2wwyWPp

Почему так: d2w — удалить два слова, w — вперёд, yw — скопировать слово, P — вставить перед курсором, p — вставить после.

Предположим, что в текстовом файле записана одна единственная строка:

```
one two three four five
```

и вам нужно преобразовать её в строку

```
three four four four five
```

Какие(ой) из предложенных ниже наборов нажатий клавиш выполнят такое редактирование? В этих наборах нажатие на клавишу Esc обозначается как <Esc> (т.е. знаки "<" и ">" не несут отдельного смысла).

Примечание: во всех утверждениях имеется в виду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Выберите все подходящие ответы из списка

☒ Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ ddthree four four four five<Esc>
- ☒ d2w\$bfour four <Esc>
- ☒ d2wwyWPp
- ☐ d2wwyWpp
- ☐ x2wwyWPp
- ☐ d2dwyWPp

Рисунок 3: Вопрос 3

Вопрос 4. Замена Windows на Linux в vim

Ответ: `:%s/Windows/Linux`

Почему так: `%` — все строки, `s` — замена, без `g` — только первое вхождение в строке.

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно **только самое первое** из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. **включая** ввод ":" в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак **не нужно**.

Напишите текст

✓ Всё получилось!

`:%s/Windows/Linux`

Следующий шаг

Решить снова

Рисунок 4: Вопрос 4

Вопрос 5. Режим выделения в vim

Ответ: - В режиме выделения можно использовать d и y - Режим выделения открывается по нажатию «v» - В режиме выделения можно использовать w, e, \$ и др. - Внизу редактора горит – VISUAL – - Выйти из режима выделения можно, нажав Esc два раза

Почему так: Я открыла vimtutor и нашла раздел про Visual mode.

Мы совсем не рассказали вам про третий режим работы vim – режим **выделения (Visual)**. Предлагаем вам ознакомиться с ним самостоятельно. Например, это можно сделать во время прохождения упражнений в vimtutor, который мы настоятельно рекомендуем вам для изучения vim!

Чтобы убедиться, что вы разобрались с этим режимом работы, отметьте, пожалуйста, **все верные** утверждения из списка ниже.

Подсказка: если вы не хотите проходить *vimtutor* целиком, то можете открыть его и поиском найти слово **"Visual"**. Вы попадете в задание, прохождение которого будет вполне достаточно, чтобы выполнить это задание.

Выберите все подходящие ответы из списка

✓ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ✓ В режиме выделения можно использовать команды d (удалить) и u (скопировать)
- ✓ Режим выделения открывается из нормального режима по нажатию "v"
- ✓ В режиме выделения можно использовать команды перемещения (например, W, e, \$, и др.)
- ✓ Когда вы находитесь в режиме выделения, внизу редактора горит надпись – VISUAL – (или – ВИЗУАЛЬНЫЙ РЕЖИМ –)
- Режим выделения открывается при помощи команды :visual
- ✓ Выйти из режима выделения можно, нажав клавишу Esc два раза

[Решить снова](#)

Рисунок 5: Вопрос 5

Вопрос 6. Запуск оболочки из оболочки

Ответ: Только из набора С

Почему так: История команд принадлежит текущей оболочке. Новая оболочка получает новую пустую историю.

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попытаете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один вариант из списка

✓ Хорошая работа.

- ☐ Из наборов В и С
- ☐ Только из набора А
- ☐ Никакие команды появляться не будут
- ☐ Только из набора В
- ☒ Только из набора С

Следующий шаг

[Решить снова](#)

Рисунок 6: Вопрос 6

Вопрос 7. Работа cd и touch в скрипте

Ответ: /home/bi/file1.txt

Почему так: `cd` меняет директорию внутри скрипта. `touch` создаёт файл там, где ты находишься.

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#), [script2.sh](#).

Предположим, что вы находитесь в директории `/home/b1/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash

cd /home/bi/
touch file1.txt
cd /home/bi/Desktop/
```

Как будет выглядеть **абсолютный путь** до созданного файла `file1.txt` по окончании работы скрипта?

Выберите один вариант из списка

✔ Правильно, молодец!

- ☐ /home/bi/Documents/file1.txt
- ☐ /home/bi/Desktop/file1.txt
- ☐ Никак (файла file1.txt не будет существовать после завершения работы скрипта)
- ☒ /home/bi/file1.txt

[Решить снова](#)

Рисунок 7: Вопрос 7

Вопрос 8. Имена переменных в bash

Ответ: - `_variable` - `VARiable` - `variable_123` - `variable123`

Почему так: Имя переменной может содержать буквы, цифры и `_`, но не может начинаться с цифры и не может содержать спецсимволы.

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [variables1.sh](#), [variables2.sh](#).

Какие из представленных ниже строк **могут** быть именами переменных в bash? Выберите **все** подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

Выберите все подходящие ответы из списка

✓ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `_variable`
- ☒ `VARiable`
- ☐ `vari/able`
- ☐ `variab$$le`
- ☒ `variable_123`
- ☒ `variable123`
- ☐ `123variable`

Рисунок 8: Вопрос 8

Вопрос 9. Скрипт с двумя аргументами

Ответ:

```
#!/bin/bash
echo "Arguments are: \"$1=$1 \"$2=$2"
```

Почему так: \$1 и \$2 — аргументы скрипта. Обратный слеш нужен, чтобы вывести именно «\$1», а не значение.

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [arguments.sh](#).

Напишите скрипт на bash, который принимает на вход два аргумента и выводит на экран строку следующего вида:

```
Arguments are: $1=первый_аргумент $2=второй_аргумент
```

Например, если ваш скрипт называется `./script.sh`, то при запуске его `./script.sh one two` на экране должно появиться:

```
Arguments are: $1=one $2=two
```

а при запуске `./script.sh three four` будет:

```
Arguments are: $1=three $2=four
```

Подсказка: в случае проблем с решением задачи, обратите внимание на наши [рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✓ Всё правильно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2 echo "Arguments are: \"$1=$1 \"$2=$2"
3
4
5
6
```


Вопрос 9. Скрипт с двумя аргументами

```
#!/bin/bash  
  
var1=$1  
var2=$2  
  
echo "Script is $0, arguments are $var1 and $var2 (total $#)"
```

Рисунок 10: Вопрос 9 продолжение

Вопрос 10. Условия, которые всегда истинны

Ответ: - -z “ ” - \$# -ge 0 - -e \$0 - -s \$0

Почему так: -z “” — пустая строка, всегда истина. \$# -ge 0 — всегда. -e \$0 и -s \$0 — сам скрипт существует и не пустой.

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [branching1.sh](#)

Предположим, вы пишете скрипт на `bash` и хотите использовать в нем конструкцию `if` в следующем фрагменте:

```
if [[ ... ]]
then
    echo "True"
fi
```

Вы можете вписать вместо "*" (внутри `[[ ]]`) и не забудьте про пробелы после `[[` и перед `]]`) любое из перечисленных ниже условий. Однако мы просим вас выбрать только те из них, при которых `echo` напечатает на экран `True` вне зависимости от того, с какими параметрами был запущен ваш скрипт и какие в нем есть переменные.

Например, условие `B -eq 0` **подходит**, т.к. ноль всегда равен нулю вне зависимости от аргументов и переменных внутри скрипта и на экран будет напечатано `True`. В то же время условие `$var1 -eq 0` **не подходит**, так как в переменной `var1` как может быть записан ноль (тогда будет напечатано `True`), так его может и не быть (тогда ничего напечатано не будет).

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ \$ тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания \$ в некоторых видах заданий на Stepik.

Выберите все подходящие ответы из списка

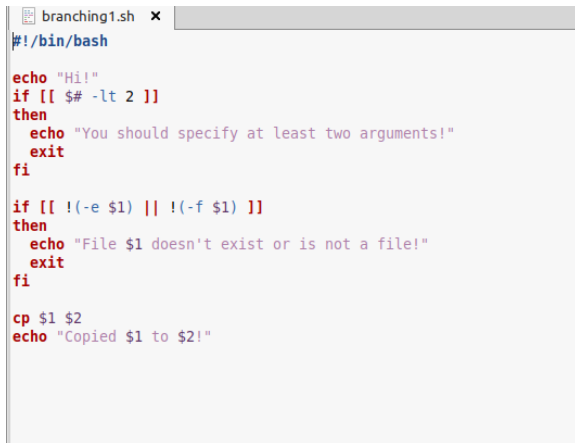
Отлично!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ $-z^{**}$
- ☒ $1/(4 - i e^3)$
- ☐ $-z^{**}$
- ☒ $-s \$0$
- ☒ $-e \$0$
- ☒ $\$e - qe 0$

[Решить снова](#)

Вопрос 10. Условия, которые всегда истинны



```
branching1.sh x
#!/bin/bash

echo "Hi!"
if [[ $# -lt 2 ]]
then
    echo "You should specify at least two arguments!"
    exit
fi

if [[ !(-e $1) || !(-f $1) ]]
then
    echo "File $1 doesn't exist or is not a file!"
    exit
fi

cp $1 $2
echo "Copied $1 to $2!"
```

Рисунок 12: Вопрос 10 продолжение

Вопрос 11. Скрипт для определения возрастной группы

Ответ: Решение в скриншоте

Почему так: Бесконечный цикл while, read запрашивает имя и возраст, проверки через if и elif.

Напишите скрипт на bash, который будет определять в какую возрастную группу попадают пользователи. При запуске скрипт должен вывести сообщение **"enter your name"** и ждать от пользователя ввода имени (используйте `read`, чтобы прочитать его). Когда имя введено, то скрипт должен написать **"enter your age:"** и ждать ввода возраста (опять нужен `read`). Когда возраст введен, скрипт пишет на экран **"<Имя>, your group is <группа>"**, где **<группа>** определяется на основе возраста по следующим правилам:

- младше либо равно 16: **"child"**,
- от 17 до 25 (включительно): **"youth"**,
- старше 25: **"adult"**.

После этого скрипт опять выводит сообщение **"enter your name:"** и всё начинается по новой (бесконечный цикл). Если в какой-то момент работы скрипта будет введено **пустое имя** или **возраст 0**, то скрипт должен написать на экран **"bye"** и закончить свою работу (выход из цикла).

Примеры корректной работы скрипта:

№1

```
./script.sh
enter your name:
Egor
enter your age:
16
Egor, your group is child
enter your name:
Elena
enter your age:
8
bye
```

№2:

```
./script.sh
enter your name:
Elena Petrovna
enter your age:
25
Elena Petrovna, your group is youth
enter your name:

bye
```

Рисунок 13: Вопрос 11

Вопрос 11. Скрипт для определения возрастной группы



Хорошие новости, верно!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свое решение с другими на [форуме решений](#).

```
1 #!/bin/bash
2  while true
3  do
4      echo "enter your name:"
5      read name
6      if [[ -z $name ]]
7      then echo "bye"
8          break
9      fi
10     echo "enter your age:"
11     read age
12     if [[ $age -eq 0 ]]
13     then echo "bye"
14         break
15     elif [[ $age -ge 0 ]] && [[ $age -le 16 ]]
16     then
17         echo "$name, your group is child"
18     elif [[ $age -ge 26 ]]
19     then
20         echo "$name, your group is adult"
21     elif [[ $age -ge 17 ]] && [[ $age -le 26 ]]
22     then
23         echo "$name, your group is youth"
24     fi
25 done
26
27
28
29
```

Следующий шаг

Решить снова

Рисунок 14: Вопрос 11 продолжение

Вопрос 12. Инструкции, увеличивающие a на b

Ответ: - let a=a+b - let «a+=b»

Почему так: a=a+b — просто строка. a+=b — не работает. let «a+=b» — правильно.

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [math1.sh](#), [math2.sh](#).

Какие(ая) из предложенных ниже инструкций увеличат значение переменной `a` на значение переменной `b`? Например, если в `a` было записано 10, в `b` было 5, то в `a` должно записаться 15.

Выберите **все подходящие** варианты!

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Подсказка: обратите особое внимание на кавычки и **пробелы**, они могут как принципиально изменить команду, так и ни на что не повлиять (в зависимости от команды и контекста)!

Выберите все подходящие ответы из списка

☒ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ a=\$a+\$b
- ☐ a+=b
- ☒ let a=a+b
- ☒ let "a+=b"
- ☐ let a = a + b

Следующий шаг

Решить снова

Рисунок 15: Вопрос 12

Вопрос 13. Что выведет echo «pwd»

Ответ: /home/bi

Почему так: Обратные кавычки выполняют pwd. cd уже сменил директорию на /home/bi/.

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [programs.sh](#).

Пусть вы находитесь в директории /home/bi/Documents/ и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash  
  
cd /home/bi/  
echo "`pwd`"
```

Что в этом случае выведет команда echo на экран?

Выберите один вариант из списка

☒ Абсолютно точно.

- ☐ Код возврата команды pwd (0 в случае успешного выполнения и не 0 в случае ошибок)
- ☐ `pwd`
- ☐ /home/bi/Documents
- ☐ pwd
- ☒ /home/bi

Следующий шаг

Решить снова

Рисунок 16: Вопрос 13

Вопрос 13. Что выведет echo «pwd»

```
#!/bin/bash

# First argument -- dir with files
# Second argument -- backup dir for these files

return_code=0
for file in `ls $1`
do
    echo "Found $file"
    if `rm $2/${file} 2>>$2/err.txt`
    then
        echo "Backup removed!"
    else
        echo "Can't remove backup!"
        return_code=1
    fi
done

exit $return_code
```

Рисунок 17: Вопрос 13 продолжение

Вопрос 14. Как проверить код возврата

Ответ: Сначала запустить program, затем if [[\$? -eq 0]]

Почему так: \$? хранит код возврата последней выполненной команды.

Мы рассказали, что можно проверить код возврата внешней программы прямо в конструкции `if` при помощи `if `program options arguments`` (действия внутри `if` выполняются, если программа закончилась с кодом 0). Однако это не всегда правда! Если запуск внешней программы выводит что-то в `stdout`, то в проверку `if` поступит именно этот вывод, а не код возврата! Вы можете убедиться в этом, написав простой `bash`-скрипт с использованием, например, `if `pwd``.

Однако как быть, если хочется все-таки запустить программу `progan`, которая пишет что-то в `stdout` и потом выполнить какие-то действия если ее код возврата равен 0? Выберите **все верные** утверждения или правильно работающие конструкции `if`.

Примечание: во всех вариантах ответов, где есть кавычка, используется именно косая кавычка ('), а не обычная (") или двойная (").

Выберите все подходящие ответы из списка

✔ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Сначала `var='program'`, затем `if [[ $var -eq 0 ]]`
- ☒ `if 'program' > some_file.txt`
- ☐ `if [[ 'program' -eq 0 ]]`
- ☐ Ничего сделать нельзя
- ☒ Сначала запустить `program`, затем `if [[ $? -eq 0 ]]`

[Решить снова](#)

Рисунок 18: Вопрос 14

Вопрос 15. Глобальные и локальные переменные

Ответ: counters are 55 and 110

Почему так: c1 — сумма чисел от 1 до 10 = 55. c2 — сумма удвоенных чисел = $2 \cdot 55 = 110$.

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [functions1.sh](#), [functions2.sh](#).

Посмотрите на функцию из bash-скрипта:

```
counter () # takes one argument
{
    local let "c1+=1"
    let "c2+=${1}*2"
}
```

Впишите в форму ниже **строку**, которую выведет на экран команда `echo "counters are $c1 and $c2"` если она находится в скрипте **после десяти вызовов** функции `counter` с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Подсказка: этот пример можно решить в уме, но если система проверки не принимает ваше решение, то возможно вы что-то упустили (возможно что-то совсем небольшое/невидимое 😊). В этом случае имеет смысл написать небольшой скрипт на bash, который проделает ровно то, что указано в задании и посимвольно сверить свой ответ с тем, что он выдаст на экран.

Напишите текст

✓ Абсолютно точно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свое решение с другими на [форуме решений](#).

counters are 55 and 110

Рисунок 19: Вопрос 15

Вопрос 16. Нахождение НОД

Ответ: Решение в скриншоте

Почему так: Функция `gcd` вызывает сама себя. При $M \neq N$ из большего вычитаем меньшее. При пустом вводе — «bye».

Напишите скрипт на `bash`, который будет искать наибольший общий делитель (НОД, *greatest common divisor*, `GCD`) двух чисел. При запуске ваш скрипт не должен ничего писать на экран, а просто ждать ввода двух натуральных чисел через пробел (для этого можно использовать `read` и указать ему две переменные — см. пример в видеофрагменте). После ввода чисел скрипт считает их НОД и выводит на экран сообщение "**GCD is «посчитанное значение»**", например, для чисел 15 и 25 это будет "GCD is 5". После этого скрипт опять входит в режим ожидания двух натуральных чисел. Если в какой-то момент работы пользователь ввел вместо этого пустую строку, то нужно написать на экран "**bye**" и закончить свою работу.

Вычисление НОД несложно реализовать с помощью [алгоритма Евклида](#). Вам нужно написать функцию `gcd`, которая принимает на вход два аргумента (назовем их **M** и **N**). Если аргументы равны, то мы нашли НОД — он равен **M** (или **N**), нужно вывести соответствующее сообщение на экран (см. выше). Иначе нужно сравнить аргументы между собой. Если **M** больше **N**, то запускаем ту же функцию `gcd`, но в качестве первого аргумента передаем (**M-N**), а в качестве второго **N**. Если же наоборот, **M** меньше **N**, то запускаем функцию `gcd` с первым аргументом **M**, а вторым (**N-M**).

Пример корректной работы скрипта:

```
./script.sh
18 15
GCD is 3
7 3
GCD is 1
bye
```

Примечание: в вызове функции из себя самой нет ничего страшного или неправильного, т.ч. смело вызывайте `gcd` прямо внутри `gcd`!

Примечание 2: для завершения работы функции в произвольном месте, можно использовать инструкцию `return` (все инструкции функции после `return` выполняться не будут). В отличие от `exit` эта команда завершит только функцию, а не выполнение всего скрипта целиком. Однако в данной задаче можно обойтись и без использования `return`!

Подсказка: в случае проблем с решением задачи, обратите внимание на наши [рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через `stdin → stdout`

Рисунок 20: Вопрос 16

Вопрос 16. Нахождение НОД

Напишите программу. Тестируется через stdin → stdout

✓ Верно. Так держаты!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 gcd () # M & N
2 {
3     if [[ -z $M && -z $N ]]; then
4         echo 'bye'
5         exit
6     fi
7
8     if [[ $M -eq $N ]]; then
9         echo "GCD is $M"
10    else
11        if [[ $M -gt $N ]]; then
12            let "M-= $N"
13            gcd $M $N
14        else
15            let "N-= $M"
16            gcd $M $N
17        fi
18    fi
19 }
20
21 while [[ true ]]; do
22     read M N
23     gcd $M $N
24 done
```

Рисунок 21: Вопрос 16 продолжение

Вопрос 17. Калькулятор

Ответ: Решение в скриншоте

Почему так: Бесконечный цикл, case обрабатывает операции, «exit» выводит «bye» и выходит, иначе «error».

Напишите **калькулятор** на bash. При запуске ваш скрипт должен ожидать ввода пользователем команды (при этом на экран выводить ничего не нужно). Команды могут быть трех типов:

1. Слово **"exit"**. В этом случае скрипт должен вывести на экран слово **"bye"** и завершить работу.
2. **Три аргумента через пробел** – первый операнд (целое число), операция (одна из "+", "-", "*", "/", "%", "**") и второй операнд (целое число). В этом случае нужно произвести указанную операцию над заданными числами и вывести результат на экран. После этого переходим в режим ожидания новой команды.
3. **Любая другая команда** из одного аргумента или из трех аргументов, но с операцией не из списка. В этом случае нужно вывести на экран слово **"error"** и завершить работу.

Чтобы проверить работу скрипта, вы можете записать сразу несколько команд в файл и передать его скрипту на stdin (т.е. выполнить `./script.sh < input.txt`). В этом случае он должен вывести сразу все ответы на экран.

Например, если входной файл будет следующего содержания:

```
10 + 1
2 ** 10
exit
```

то на экране будет:

```
11
1024
bye
```

Рисунок 22: Вопрос 17

Вопрос 17. Калькулятор



Абсолютно точно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить с

```
1 #!/bin/bash
2 while [[ True ]]
3 do
4     read birinchi amal ikkinchi
5     if [[ $birinchi == "exit" ]]
6     then
7         echo "bye"
8         break
9     elif [[ "$birinchi" =~ "^[0-9]+$" && "$ikkinchi" =~ "^[0-9]+$" ]]
10    then
11        echo "error"
12        break
13    else
14        case $amal in
15        "+") let "result = birinchi + ikkinchi";;
16        "-") let "result = birinchi - ikkinchi";;
17        "/" let "result = birinchi / ikkinchi";;
18        "*" let "result = birinchi * ikkinchi";;
19        "%" let "result = birinchi % ikkinchi";;
20        "**") let "result = birinchi ** ikkinchi";;
21        *) echo "error" ; break ;;
22        esac
23        echo "$result"
24    fi
25 done
26
27
```

Вопрос 18. Поиск с -iname и -name

Ответ: - Star_Wars.avi - STARS.txt

Почему так: -iname не чувствительна к регистру, -name чувствительна.

Пусть в директории `/home/bi` лежат файлы `Star_Wars.avi`, `star_trek_OST.mp3`, `STARS.txt`, `stardust.mpeg`, `Eddard_Stark_biography.txt`.

Отметьте все файлы, которые **найдет** команда `find /home/bi -iname "star*"`, но **НЕ найдет** команда `find /home/bi -name "star*" ?`

Выберите все подходящие ответы из списка

☒ Прекрасный ответ.

- ☐ star_trek_OST.mp3
- ☐ stardust.mpeg
- ☐ Eddard_Stark_biography.txt
- ☒ STARS.txt
- ☒ Star_Wars.avi

Следующий шаг

Решить снова

Рисунок 24: Вопрос 18

Вопрос 19. Сравнение -name и -path

Ответ: - В некоторых случаях find с -name найдёт больше файлов, чем с -path - Если заменить -name на -path, результат иногда может остаться таким же

Почему так: -path ищет по всему пути, -name — только по имени файла.

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте все верные утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

✓ Прекрасный ответ.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ В некоторых случаях find с -name найдет больше файлов, чем find с таким же запросом, но с -path
- ☒ В некоторых случаях find с -name найдет меньше файлов, чем find с таким же запросом, но с -path
- ☐ Опция -path аналогична -name, но игнорирует размер букв (строчные/прописные) в имени файла
- ☐ Если заменить в команде поиска -name, на -path, то результат поиска всегда останется неизменным
- ☒ Если заменить в команде поиска -name, на -path, то результат поиска иногда может остаться таким же

Следующий шаг

[Решить снова](#)

Рисунок 25: Вопрос 19

Вопрос 20. Поиск с -mindepth и -maxdepth

Ответ: Ни один файл найден не будет

Почему так: file1, file2, file3 лежат на глубине 1, а диапазон 2-3.

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:

```

/home/b1/
├── dir1
│   ├── file1
│   └── dir2
│       ├── file2
│       └── dir3
│           └── file3

```

Какие(ой) из трех файлов (file1, file2, file3) будут найдены по команде `find /home/b1 -mindepth 2 -maxdepth 3 -name "file*"` ?

Выберите один вариант из списка

✓ Абсолютно точно.

- ☐ Ни один файл найден не будет
- ☐ Только file3
- ☐ Все кроме file2
- ☒ Все кроме file3
- ☐ Все кроме file1

[Решить снова](#)

Рисунок 26: Вопрос 20

Вопрос 21. Опции grep -A, -B, -C

Ответ: `grep -C 1 «word» file.txt > results.txt`

Почему так: `-C 1` выводит строку + строку до и после. Это больше, чем `-A 1` или `-B 1`.

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем в каждой строке есть слово `"word"`. Если вы выполните на этом файле команды:

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один вариант из списка

☒ Здорово, всё верно.

- ☐ `grep -C 1 "word" file.txt > results.txt`
- ☐ Все, кроме `grep "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt` и `grep -B 1 "word" file.txt > results.txt`
- ☒ `results.txt` будет одинакового размера во всех случаях

Следующий шаг

Решить снова

Рисунок 27: Вопрос 21

Вопрос 22. Регулярное выражение в grep

Ответ: - I prefer Kubuntu - Hmm, XKUbuntu - The best OS is Xubuntu

Почему так: Шаблон ищет строки, заканчивающиеся на «buntu», перед которым может быть и или символ из [xKXKL].

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xKXKL]?[uU]buntu$" text.txt`.

Выберите все подходящие ответы из списка

✓ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свое решение с другими на [форуме решений](#).

- ☒ Mac OS X, Windows, Ubuntu
- ☒ I prefer Kubuntu
- ☒ Hmm, XKUbuntu
- ☒ Linux is not always Ubuntu
- ☒ The best OS is Xubuntu
- ☐ Uuuubuntu!

Следующий шаг

Решить снова

Вы получили: 2 балла

Верно решили 28 165 учас

Рисунок 28: Вопрос 22

Вопрос 23. Замена «аббревиатур» в sed

Ответ: `sed -E 's/([A-Z]{2,})/abbreviation/g' input.txt > edited.txt`

Почему так: `[A-Z]{2,}` — две или больше заглавные буквы.

Запишите в форму ниже инструкцию `sed`, которая заменит все «аббревиатуры» в файле `input.txt` на слово «abbreviation» и запишет результат в файл `edited.txt` (на экран при этом ничего выводить не нужно). Обратите внимание, что **в инструкции должны быть указаны и сам `sed`, и оба файла!**

Под «аббревиатурой» будем понимать слово, которое удовлетворяет следующим условиям:

- состоит только из больших букв латинского алфавита,
- состоит из хотя бы двух букв,
- окружено одним пробелом с каждой стороны.

При этом будем считать, что в тексте **не может быть две «аббревиатуры» подряд**. Например, текст `" YOU YOU and YOU!"` является **некорректным** (в нем есть две «аббревиатуры», но они идут подряд) и на таких примерах мы проверять вашу инструкцию **не будем**.

Пример: если у вас был текст `"Hi, I heard these songs by ABBA, TLA and DM!"`, то он должен быть преобразован в `"Hi, I heard these songs by ABBA, abbreviation and abbreviation!"`.

Примечание: после вашей замены «аббревиатур» на слово «abbreviation» **количество пробелов** в тексте **не должно меняться!**

Внимание! Во время проверки **мы не запускаем команду**, которую вы ввели на реальном файле с «аббревиатурами» (это небезопасно, можно же ввести `rm -rf /`!). Вместо этого мы сперва анализируем структуру вашей инструкции (например, что в ней использован именно `sed` и сделано это ровно один раз, что на вход подается `input.txt`, а результат будет записан в `edited.txt` и т.д.), а затем **запускаем её смысловую часть** (т.е. поиск по регулярному выражению и замена на «abbreviation») на тестовых примерах. К сожалению, наш запуск не идеально повторяет `sed`, но он очень близок к нему. Главная «несовместимость» заключается в том, что наша проверка не понимает идущие подряд символы, отвечающие за количество повторов (т.е. `*`, `+`, `?` и `{}`). Однако эту «несовместимость» легко исправить указав при помощи `[]` и `{}` какой из символов к чему относится! Например, регулярное выражение `at?` (ноль или один раз по одной или более букве «a») нужно записать как `(a+)?` (при этом запись `(a)?+`, конечно же, не поможет).

Напишите текст

☒ Так точно!

```
sed -E 's/([A-Z]{2,})/abbreviation/g' input.txt > edited.txt
```

Рисунок 29: Вопрос 23

Вопрос 24. Опция -p в gnuplot

Ответ: -p --persist

Почему так: Без -p графики закрываются вместе с gnuplot. С -p остаются открытыми.

Вы можете скачать и попробовать применить gnuplot к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске gnuplot, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один вариант из списка

☒ Верно.

- ☐ Такой опции не существует
- ☐ -s, --show-plots-after-exit
- ☒ -p, --persist
- ☐ Графики и так не закрываются автоматически при закрытии gnuplot!

Следующий шаг

Решить снова

Рисунок 30: Вопрос 24

Вопрос 25. Название ряда в gnuplot

Ответ: Название — первое значение из второго столбца, нарисовано 9 точек

Почему так: autotitle columnhead берёт заголовок из первой строки. Первая строка не рисуется, остаётся 9 точек.

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете `gnuplot` и вводите в него две команды:

```
set key autotitle columnhead
plot 'data.csv' using 1:2
```

Какое в этом случае будет название **y** построенного ряда данных и сколько будет нарисовано **точек** на графике?

Выберите один вариант из списка

✓ Хорошая работа.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [фотоше решения](#).

- ☐ Название "popate", нарисовано 10 точек
- ☐ Название – первое значение из первого столбца, нарисовано 10 точек
- ☒ Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)
- ☐ Название – первое значение из второго столбца, нарисовано 10 точек
- ☐ Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)

[Решить снова](#)

Рисунок 31: Вопрос 25

Вопрос 26. Установка меток на оси в gnuplot

Ответ: `set xtics («point 1, value».x1, «point 2, value».x2, «point 3, value».x3)`

Почему так: Конкатенация строк через “. Без кавычек gnuplot подставит значения переменных.

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [plot.gnu](#), [plot_advanced.gnu](#), [plot_advanced2.gnu](#). Все три скрипта основаны на [этой заметке](#), данные также взяты оттуда.

Предположим, что вы пишете поли-скрипт и у вас в нем есть три переменные x_1 , x_2 , x_3 , которые записаны координаты различных точек по оси ОХ (по возрастанию). Вы хотите, чтобы на этой оси было только три деления (т.е. три черточки) и эти самых координатах, а подписи этих делений были оформлены в виде **"point-номер точки"**, **value-значение соответствующей переменной**. Например, для $x_1=8$, $x_2=18$, $x_3=28$, это были бы надписи "point 1, value 0" в точке с координатой 0 по горизонтали, "point 2, value 10" в точке с координатой 10 и "point 3, value 20" в точке с координатой 20. Или, например, $x_1=100$, $x_2=150$, $x_3=250$, это были бы надписи "point 1, value 100" в точке с координатой 100, "point 2, value 150" в точке с координатой 150 и "point 3, value 250" в точке с координатой 250.

Впишите в форму ниже **одну команду** (т.е. одну строку), которую нужно добавить в скрипт, для выполнения этой задачи.

Примечание: проверять, что переменные `x1`, `x2`, `x3` идут по возрастанию или что они являются числами не нужно

Примечание 2: в видеофрагменте на предыдущем шаге звучал термин конкатенация, который важен для выполнения данного задания. Под конкатенацией обычно понимают "склеивание" двух строк в одну длинную строку, например, конкатенация строк "Данные из файла " и "data.csv" даст строку "Данные из файла data.csv".

Подсказка: настоятельно рекомендуем изучить примеры скриптов – в них есть большая часть решения!

Напишете текст

✓ Верно. Так держать!

```
set xtics ("point 1, value " x1 x1, "point 2, value " x2 x2, "point 3, value " x3 x3)
```

Рисунок 32: Вопрос 26

Вопрос 27. Изменение анимации в gnuplot

Ответ:

```
a=a+1
zrot=(zrot+350)%360
set view xrot,zrot
splot -x**2-y**2
pause 0.1
if (a<50) reread
```

Почему так: `zrot=(zrot+350)%360` — вращение в другую сторону. `pause 0.1` вместо `0.2` — скорость увеличилась.

Если вы не скачали на предыдущем шаге файлы [animated.gnu](#) и [move.rot](#), то скачайте их теперь, т.к. они понадобятся для выполнения задания.

Указанные файлы использовались в последнем видеофрагменте для создания вращающегося графика. Измените инструкции в файле `move.rot` (т.е. **добавлять и удалять инструкции нельзя!**) таким образом, чтобы:

- График **отразился зеркально** относительно горизонтальной поверхности. То есть там, где была точка (10, 10, 200), станет точка (10, 10, -200), где была точка (-10, -10, 200) станет (-10, -10, -200) и т.д. При этом точка (0, 0, 0) останется на месте.
- Изображение стало **вращаться в обратную сторону**. То есть если раньше вращалось "влево", то теперь станет "вправо".
- Вращение стало **в два раза быстрее**. То есть станет в два раза больше перерисовок графика на каждую секунду вращения.

Измененный файл загрузите в форму ниже.

Примечание: наша система проверки **не может** запустить на вашем файле `move.rot` программу `gnuplot` и сравнить полученный график с заданным. Вместо этого **мы анализируем команды**, которые вы указали в файле. Поэтому если вы видите, что ваш скрипт в `gnuplot` работает точно по условию, а мы отвечаем "Incorrect/Неверно", то попробуйте упростить свою модификацию `move.rot` и отправить его еще раз.

Вопрос 28. Права доступа rwxrw-r—

Ответ: `chmod u+wx file.txt; chmod g+w file.txt`

Почему так: `u+wx` — добавляет владельцу `w` и `x`. `g+w` — добавляет группе `w`.

Какая команда(ы) установят файлу `file.txt` права доступа `rwxrw-r--`, если изначально у него были права `r--r--r--`. Укажите **все** верные варианты ответа!

Примечание: запись вида `команда1; команда2; команда3` означает, что в терминале последовательно выполнялись все три команды (сначала `команда1`, затем `команда2` и, наконец, `команда3`).

Выберите все подходящие ответы из списка

☒ Отличное решение!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свое решение с другими на [форуме решений](#).

- ☐ `chmod u-wx file.txt; chmod g-w file.txt`
- ☐ `chmod 467 file.txt`
- ☒ `chmod u+wx file.txt; chmod g+w file.txt`
- ☐ `chmod rwxrw-r-- file.txt`
- ☐ `chmod o-wx file.txt; chmod g-x file.txt; chmod a+wx file.txt`
- ☐ `chmod 777 file.txt`

Следующий шаг

Решить снова

Рисунок 34: Вопрос 28

Вопрос 29. Создание файла в директории root

Ответ: - sudo chown user dir - sudo chown :group dir

Почему так: Если директория принадлежит root, а группа правильная, то пользователь получит права группы и сможет писать.

Предположим вы использовали команду `sudo` для создания директории `dir`. По умолчанию для `dir` были выставлены права доступа `rxwx-xr-x` (адаплец `root`, группа `root`). Таким образом никто кроме пользователя `root` не может ничего записывать в эту директорию, например, не может создавать файлы в ней.

После выполнения какой команды **user** из группы **group** всё-таки сможет создать файл внутри **dir** ? Укажите **все верные** варианты ответов!

Примечание: считаем, что все команды выполняются от имени `user`, если явно не указано, что команда выполнена с `sudo`

Примечание 2: мы выбрали пример с директорией, а не с файлом не случайно

Дело в том, что если создать при помощи `sudo` файл с правами `rw-r--r--` в директории, которая принадлежит пользователю, то возникает любопытная ситуация. С одной стороны пользователь может удалить этот файл (т.к. ему разрешено **все** файлы внутри его директории) и может прочитать его содержимое (т.к. право `rw` у файла установлено для **всех**), с другой стороны он не может этот файл редактировать (т.к. право `w` у файла есть только для **root**). При этом некоторые "умные" редакторы, например, `vim` позволяют даже редактировать этот файл, но делают они это своеобразно: через удаление оригинала и создание копии уже с нужными правами (удалить мы можем, а раз можем читать, то и копию сделать не сложно). Итого получается, что несмотря на права `rw-r--r--`, пользователь может сделать с этим файлом почти все что угодно!

В случае же, когда речь идет о директории созданной **root**, ситуация будет проще: пользователь сможет посмотреть её содержимое (у него есть право **"r"**), но удалять и создавать файлы в ней не сможет (права **"w"** у него нет).

Важно отметить, что директорию в *Linux* это в каком-то смысле файлы. Содержимое такого "файла" – это записи о файлах и поддиректориях этой директории (грубо говоря их название). Таким образом, право "r" у директории дает возможность просматривать "записи", т.е. просматривать её состав. Право "w" у директории дает возможность удалять/добавлять новые "записи", т.е. удалять/создавать файлы/поддиректории в ней.

На самом деле и это еще не всё. Существует так называемый **sticky bit** (атрибут файла или директории), выставление которого меняет описанное выше поведение. Файлы (или директории) с таким атрибутом сможет удалить только их владелец вне зависимости от прав, установленных у директории, в которой эти файлы (или директории) лежат!

Отдельное спасибо слушателю курса **Alexey Antipovsky** за помощь в оформлении Примечания 2!

Выберите все подходящие ответы из списка

✔ Все правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свой решение с другими на [форуме решений](#).

- ☐ sudo chmod o+x dir
- ☐ sudo chown :group dir
- ☐ chmod o+w dir
- ☐ chown user:group dir
- ☒ sudo chown user dir

Вопрос 30. Команда du для размера текущей директории

Ответ: du -hs

Почему так: du -h — человеко-читаемый формат. du -s — суммарный размер.

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести **в удобном для чтения формате** (например, вместо 2048 байт надо выводить 2.0K) и **больше** на экран выводить **ничего не** нужно). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по 800 Кбайт и две поддиректории в каждой из которой лежит по файлу в 400 Кбайт, то загаданная команда должна вывести на экран одно число: 2.4M (также на экране может быть выведен еще и символ ".", обозначающий, что это размер именно текущей директории).

Напишите текст

✓ Всё правильно.

```
du -h -s
```

Следующий шаг

Решить снова

Рисунок 36: Вопрос 30

Вопрос 31. Создание трёх поддиректорий

Ответ: `mkdir dir{1..3}`

Почему так: `{1..3}` разворачивается в `1 2 3`. Получается `mkdir dir1 dir2 dir3`.

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст

✓ Отличное решение!

```
mkdir dir{1..3}
```

Рисунок 37: Вопрос 31

Вопрос 32. Опция -n в sed

Ответ: На экран будет выведено всё содержимое файла

Почему так: Без -n sed печатает все строки по умолчанию. Команда p не меняет этого.

Что произойдет, если в команде `sed -n "/[a-z]*/p" text.txt` не указывать опцию `-n` ?

Выберите один вариант из списка



Хорошая работа.

- ☐ На экран ничего не напечатается
- ☐ На экран будет выведено всё содержимое файла text.txt
- ☒ Каждая строка будет выведена два раза
- ☐ Появится сообщение об ошибке

Следующий шаг

Решить снова

Рисунок 38: Вопрос 32

Раздел 2

2. ВЫВОДЫ

Что я научилась делать

- Работать в vim (перемещение, замена, режимы)

Все задания третьего раздела выполнены.

Что я научилась делать

- Работать в vim (перемещение, замена, режимы)
- Писать bash-скрипты (переменные, условия, циклы, функции, арифметика)

Все задания третьего раздела выполнены.

Что я научилась делать

- Работать в vim (перемещение, замена, режимы)
- Писать bash-скрипты (переменные, условия, циклы, функции, арифметика)
- Обрабатывать текст с помощью sed

Все задания третьего раздела выполнены.

Что я научилась делать

- Работать в vim (перемещение, замена, режимы)
- Писать bash-скрипты (переменные, условия, циклы, функции, арифметика)
- Обрабатывать текст с помощью sed
- Строить графики в gnuplot

Все задания третьего раздела выполнены.

